

Wallet et retraits Mobile Money — guide d'intégration mobile

Ce guide couvre le parcours complet côté application : consulter le solde, enregistrer un compte Mobile Money, demander un retrait, valider le code reçu par SMS, suivre le paiement.

Il tient compte de l'ouverture du module au **Sénégal** et au **Congo**, qui change les valeurs acceptées sans changer un seul nom de champ.

Tous les exemples de ce document sont des échanges réels avec le backend, pas des maquettes.

1. Ce qui change pour l'application

Aucun paramètre ne change. Ni nom de champ, ni type, ni forme de réponse. Une application qui envoie déjà `operator` , `phoneNumber` et `accountName` continue de fonctionner.

Trois points demandent malgré tout une mise à jour, sans quoi des utilisateurs seront bloqués sans comprendre pourquoi.

À revoir	Pourquoi
La liste des moyens de paiement proposée	Proposer Moov ou Celtis à un Béninois produit désormais une erreur 400
La validation locale du numéro	Une règle codée en dur sur <code>+229</code> bloquerait un Sénégalais avant même l'appel
Le code 409 à l'enregistrement du compte	Nouveau cas, avec un message explicite à afficher tel quel

2. Pays, numéros et moyens de paiement

Le moyen de paiement décrit **par où l'argent sort**, pas le réseau de la carte SIM du bénéficiaire.

Pays	Format du numéro	Moyen de paiement	Exemple
Bénin	<code>+229 01</code> + 8 chiffres	MTN_MOMO	<code>+229 0161345578</code>
Sénégal	<code>+221 7X</code> + 7 chiffres	WAVE	<code>+221 771234567</code>
Congo	<code>+242 0X</code> + 7 chiffres	MTN_MOMO	<code>+242 061234567</code>

Préfixes mobiles reconnus : **70, 75, 76, 77, 78** au Sénégal ; **04, 05, 06** au Congo-Brazzaville.

Les espaces, points et tirets sont tolérés à la saisie : `+229 01 61 34 55 78` et `+229-01.61345578` sont acceptés et normalisés en `+229 0161345578` . C'est cette forme normalisée que renvoie l'API et qu'il faut afficher.

Wave n'est pas un opérateur télécom : il fonctionne sur tous les réseaux sénégalais. C'est pourquoi **tous** les mobiles sénégalais sont acceptés, alors que le moyen de paiement, lui, est unique.

3. Enregistrer ou modifier le compte Mobile Money

`POST /user-payment/accounts`

Un utilisateur n'a **qu'un seul** compte par défaut : ce même appel crée le compte la première fois, puis le remplace ensuite. L'ancien numéro est conservé dans l'historique côté serveur.

Requête

```
{
  "operator": "MTN_MOMO",
  "phoneNumber": "+229 01 61 34 55 78",
  "accountName": "AKPOVI Jules"
}
```

Réponse — 201

```
{
  "id": "fd808393-c43c-471c-8e10-b1b7c351d109",
  "userId": 2,
  "operator": "MTN_MOMO",
  "phoneNumber": "+229 0161345578",
  "accountName": "AKPOVI Jules",
  "isDefault": true,
  "verified": false
}
```

Le champ `id` est l'identifiant à passer plus tard comme `paymentAccountId` dans la demande de retrait. `verified` repasse à `false` à chaque modification : c'est la vérification par OTP du retrait qui fait foi.

Erreurs

Format de numéro refusé — 400. Le message est une liste, comme toute erreur de validation NestJS.

```
{
  "statusCode": 400,
  "message": [
    "Le numéro Mobile Money doit respecter l'un des formats acceptés : +229 01XXXXXXX, +221 7XXXXXXX, +242 0XXXXXXX"
  ]
}
```

Moyen de paiement indisponible dans ce pays — 400. Le message est une chaîne, pas une liste : le numéro est valide, c'est le couple qui ne l'est pas.

```
{
  "message": "Le moyen de paiement MOOV_MONEY n'est pas disponible pour un numéro Bénin (+229 01XXXXXXX). Accepté : MTN_MOMO.",
  "error": "Bad Request",
  "statusCode": 400
}
```

Retrait en cours de traitement — 409. Nouveau cas.

```
{
  "message": "Une demande de retrait est en cours de traitement. Elle doit être payée ou rejetée avant de modifier votre compte Mobile Money.",
  "error": "Conflict",
  "statusCode": 409
}
```

Ce refus protège le virement : une demande déjà validée par OTP pointe sur le compte, et en changer le numéro ferait payer une destination que la vérification n'a jamais couverte. **Tant que la demande est en `OTP_PENDING`, la modification reste possible** — c'est le cas de qui s'est trompé de numéro. Le code déjà envoyé est alors périmé : il faut en demander un nouveau.

Consulter le compte enregistré

GET /user-payment/accounts/me

```
[
  {
    "id": "fd808393-c43c-471c-8e10-b1b7c351d109",
    "userId": 2,
    "operator": "MTN_MOMO",
    "phoneNumber": "+229 0148447717",
    "accountName": "Test",
    "isDefault": true,
    "verified": false
  }
]
```

4. Consulter le wallet

`GET /wallet/me` renvoie le solde et l'historique. `GET /wallet/me/overview` en donne une version allégée pour l'écran d'accueil, avec les dernières opérations financières.

```
{
  "wallet": {
    "id": "6a23cdad-f676-4973-8ad9-477ba5cce4e8",
    "userId": 2,
    "balance": 5000,
    "availableBalance": 5000,
    "pendingBalance": 0,
    "currency": "XOF",
    "status": "ACTIVE"
  },
  "transactions": []
}
```

`availableBalance` est le montant retirable. `pendingBalance` est immobilisé par une demande en cours : ne proposez jamais de retirer `balance`.

5. Demander un retrait

`POST /wallet/withdrawals`

```
{
  "amount": 1000,
  "paymentMethod": "MOBILE_MONEY",
  "paymentAccountId": "fd808393-c43c-471c-8e10-b1b7c351d109"
}
```

`paymentAccountId` doit être un UUID : c'est l' `id` renvoyé à l'étape 3. Il ne s'agit pas du numéro de téléphone.

Réponse — 201

La demande naît en `OTP_PENDING` et le SMS part immédiatement.

```
{
  "id": "15e83375-9b79-48d2-8846-5d09785560b9",
  "amount": 1000,
  "fees": 0,
  "netAmount": 1000,
  "status": "OTP_PENDING",
  "paymentMethod": "MOBILE_MONEY",
  "paymentAccountId": "fd808393-c43c-471c-8e10-b1b7c351d109",
  "otp": {
    "sent": true,
    "provider": "infobip",
    "deliveryStatus": "SENT_TO_PROVIDER",
    "expiresAt": "2026-08-13T05:09:20.279Z",
    "maxAttempts": 3,
    "debugAvailable": true,
    "failureReason": null
  },
  "message": "Votre code de validation est en cours d'envoi au numéro +229 0161345578. Si vous ne le recevez pas, vous pourrez demander un renvoi sécurisé."
}
```

Affichez `netAmount`, pas `amount` : c'est ce que la personne recevra une fois les frais retenus.

Erreurs métier — 400

Ce point d'entrée renvoie une forme particulière, avec le détail des règles :

```
{
  "message": "Retrait refusé par les règles métier",
  "errors": [
    {
      "passed": false,
      "code": "NO_PENDING_WITHDRAWAL",
      "message": "Une demande de retrait est déjà en cours"
    }
  ]
}
```

Codes à traiter en priorité :

Code	Sens
NO_PENDING_WITHDRAWAL	Une demande est déjà ouverte ; une seule à la fois
AVAILABLE_BALANCE	Solde disponible insuffisant
MIN_MAX_WITHDRAWAL	Montant hors des bornes configurées
PAYMENT_ACCOUNT_EXISTS	Aucun compte Mobile Money enregistré
BENIN_PAYMENT_ACCOUNT_PHONE	Numéro non conforme aux formats acceptés
EMAIL_VERIFIED	Adresse électronique non vérifiée
RESTRICTION_CAN_WITHDRAW	Retrait bloqué pour ce compte

Le code `BENIN_PAYMENT_ACCOUNT_PHONE` conserve son nom d'origine alors qu'il couvre désormais trois pays : le renommer aurait cassé les applications qui s'y branchent. Ne vous fiez pas au mot « BENIN » qu'il contient.

6. Valider le code OTP

POST `/wallet/withdrawals/{id}/verify-otp`

```
{ "code": "325610" }
```

Code incorrect — 400. Le décompte des tentatives est dans le message.

```
{
  "message": "Code OTP incorrect. Tentative 1/3.",
  "error": "Bad Request",
  "statusCode": 400
}
```

Code correct — 201. La demande passe en `PENDING` : elle attend désormais un administrateur.

```
{
  "id": "15e83375-9b79-48d2-8846-5d09785560b9",
  "amount": 1000,
  "netAmount": 1000,
  "status": "PENDING",
  "paymentAccountId": "fd808393-c43c-471c-8e10-b1b7c351d109"
}
```

Au-delà de trois tentatives, la demande est verrouillée et seul un administrateur peut la débloquent.

Renvoyer le code

POST `/wallet/withdrawals/{id}/resend-otp` — sans corps.

Le renvoi est limité : un délai minimal entre deux envois, et un nombre maximal de renvois par demande. Au-delà, la réponse indique le blocage ; c'est le message à afficher, il est écrit pour l'utilisateur final.

7. Suivre la demande

GET /wallet/withdrawals/current

```
{
  "hasCurrentWithdrawal": true,
  "withdrawal": {
    "id": "15e83375-9b79-48d2-8846-5d09785560b9",
    "amount": 1000,
    "netAmount": 1000,
    "status": "OTP_PENDING",
    "paymentAccountId": "fd808393-c43c-471c-8e10-b1b7c351d109"
  }
}
```

Quand aucune demande n'est ouverte, `hasCurrentWithdrawal` vaut `false` .

GET /wallet/me/transactions donne l'historique détaillé, étape par étape : création de la demande, envoi et livraison du SMS, vérification du code, approbation, paiement, débit final. C'est la source à utiliser pour un écran de suivi.

Les statuts

Statut	Ce que voit l'utilisateur
OTP_PENDING	En attente du code SMS
PENDING	Code validé, en attente de traitement
APPROVED	Approuvée, paiement en préparation
PROCESSING	Paiement en cours
PAID	Payée
REJECTED	Refusée, avec un motif
SECURITY_REVIEW_REQUIRED	Vérification de sécurité en cours
OTP_EXPIRED	Code expiré, à recommencer

Seuls PAID , REJECTED , CANCELLED , FAILED et OTP_EXPIRED libèrent la possibilité de créer une nouvelle demande.

8. Exemple Flutter

```
class PaiementApi {
  PaiementApi(this.client, this.baseUrl, this.token, this.pays);

  final http.Client client;
  final String baseUrl, token, pays;

  Map<String, String> get _headers => {
    'Authorization': 'Bearer $token',
    'Content-Type': 'application/json',
  };

  /// Enregistre ou remplace le compte Mobile Money.
  /// Renvoie l'identifiant du compte, à réutiliser pour le retrait.
  Future<String> enregistrerCompte({
    required String operateur,
    required String numero,
    required String titulaire,
  }) async {
    final reponse = await client.post(
      Uri.parse('$baseUrl/user-payment/accounts?country=$pays'),
      headers: _headers,
      body: jsonEncode({
        'operator': operateur,
        'phoneNumber': numero,
        'accountName': titulaire,
      }),
    );

    if (reponse.statusCode == 201) {
      return jsonDecode(reponse.body)['id'] as String;
    }

    // 409 : une demande de retrait validée est en cours.
    // 400 : format du numéro, ou moyen de paiement absent du pays.
    throw PaiementException(_messageDe(reponse));
  }

  Future<Map<String, dynamic>> demanderRetrait({
    required num montant,
    required String compteId,
  }) async {
    final reponse = await client.post(
      Uri.parse('$baseUrl/wallet/withdrawals?country=$pays'),
      headers: _headers,
      body: jsonEncode({
        'amount': montant,
        'paymentMethod': 'MOBILE_MONEY',
        'paymentAccountId': compteId,
      }),
    );

    if (reponse.statusCode == 201) {
      return jsonDecode(reponse.body) as Map<String, dynamic>;
    }
    throw PaiementException(_messageDe(reponse));
  }

  /// Les messages du serveur sont rédigés pour l'utilisateur final :
  /// les afficher tels quels vaut mieux que les reformuler.
  String _messageDe(http.Response reponse) {
    final corps = jsonDecode(reponse.body) as Map<String, dynamic>;

    // Règles métier du retrait : liste de codes détaillés.
    final regles = corps['errors'] as List?;
    if (regles != null && regles.isNotEmpty) {
      return regles.first['message'] as String;
    }

    // Validation de DTO : message sous forme de liste.
    final message = corps['message'];
    if (message is List && message.isNotEmpty) return message.first as String;
    if (message is String) return message;
  }
}
```

```
    return 'Une erreur est survenue.';
  }
}
```

Choisir le moyen de paiement selon le pays

```
/// À aligner sur le pays du compte, sinon l'API répond 400.
const moyensParPays = {
  'benin': ['MTN_MOMO'],
  'senegal': ['WAVE'],
  'congo': ['MTN_MOMO'],
};

/// Validation locale – doit rester cohérente avec le serveur.
final formatsNumero = RegExp(
  r'^\+(?:229[\s.-]?01(?:[\s.-]?\\d){8}'
  r'|221[\s.-]?7[05678](?:[\s.-]?\\d){7}'
  r'|242[\s.-]?0[456](?:[\s.-]?\\d){7})$',
);
```

9. Rappels

Tous ces points d'entrée exigent le jeton JWT et le paramètre `?country=`, comme le reste de l'API.

Un seul retrait à la fois. Vérifiez `GET /wallet/withdrawals/current` avant d'ouvrir l'écran de demande : cela évite un 400 inutile.

Ne codez pas les frais en dur. `fees` et `netAmount` sont calculés par le serveur selon la configuration en vigueur.

Le numéro affiché est celui renvoyé par l'API, sous sa forme normalisée. Ne le reformatez pas côté application, au risque de le rendre méconnaissable.

Les messages d'erreur sont rédigés pour l'utilisateur final. Les afficher tels quels donne une meilleure information qu'un texte générique — en particulier pour le 409 et pour les codes de règles métier.